

Twin-engine balsa RC aircraft, designed in SolidWorks and laser cut, with its transmitter.

Alden Roswell

Engineering Portfolio

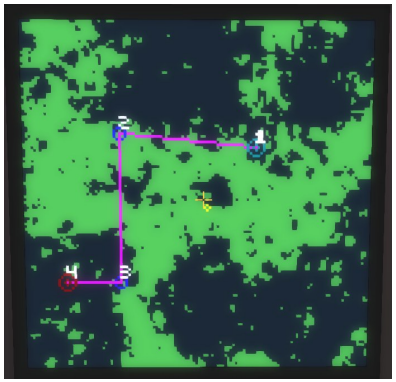
roswellalden@gmail.com | (720) 695-9991 | Lafayette, CO
LinkedIn | Github

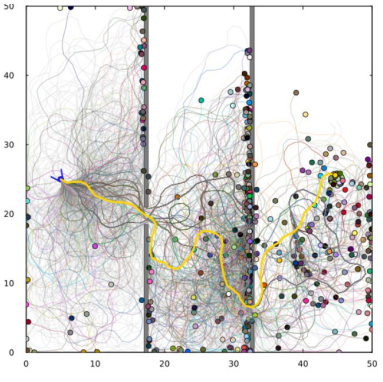
Aerospace Engineering senior at CU Boulder, minor in Computer Science, focused on guidance, navigation, and control. What I enjoy most is where computation, analysis, and design meet: predicting how something will behave, building it, and closing the gap when it doesn't match.

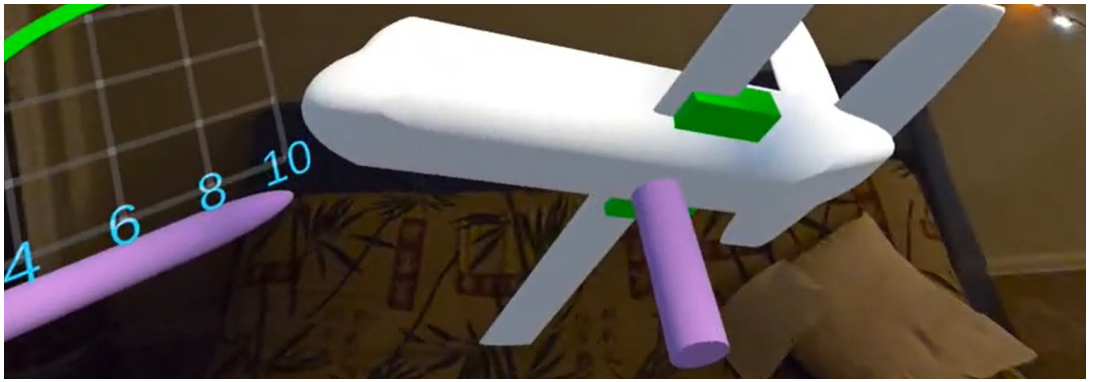
My professional GNC analysis at Space Kinetic and hardware work at Red 6 can't be shown here. My resume covers both.

Contents

2 Autonomous Flight Controller
3 Kino-PASTA
4 XR Design & Fly
5 RC Aircraft



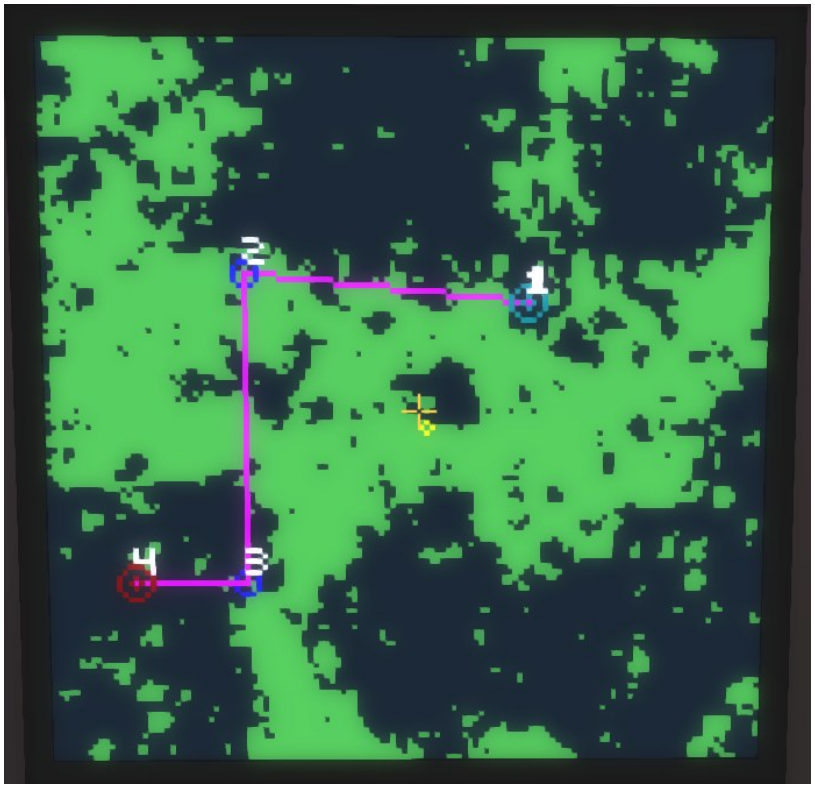




Autonomous Flight Controller

Oct 2023 – Jan 2024 | Personal project

Waypoint guidance, cascaded control, and drop prediction, written in Lua in Scrap Mechanic



Ground terminal display: terrain map from a raycast survey, with numbered waypoints and the planned route.

Video: youtu.be/ow1i8SoXi5U
Full end-to-end autonomous flight.
Code: GitHub repository

Working around the sandbox

- The terrain image was too large to store as a variable, so I encoded it as a string of ones and zeros embedded directly in the script.
- Raycasts don't compute outside the player's range, so the survey script spawns a character and teleports it to each pixel before casting.

A computer mod for Scrap Mechanic lets you script vehicles in Lua. I built an aircraft that flies a flight plan from a ground terminal, drops a payload on a target coordinate, and lands. Each lifting surface runs its own script that computes angle of attack and applies lift and drag, with lift cut off past about 22 degrees to represent stall.

Guidance

- Flight plans are typed into the terminal and sent to the aircraft as JSON packets. The aircraft switches waypoints at 250 m.
- Heading error to the next waypoint becomes a bank command, and altitude error becomes a pitch command.
- For landing, the terminal generates an approach fix 1.5 km out on the extended centerline and a 1:7 glide slope, plus the threshold and runway end. On approach the aircraft tracks the glide slope, steers on cross-track error, and slows with a tighter bank limit, then cuts throttle and brakes past the threshold.
- To time the drop, the aircraft integrates the payload's trajectory forward at the game's 40 Hz tick. I logged a falling object's velocity every tick and back-calculated the game's per-tick drag decay so the prediction matched. Release happens when predicted ground distance matches distance to target.

Navigation

The game provides true state, so there was no estimation problem to solve. I derived angle of attack, sideslip, attitude, and body-axis velocities from it. A raycast survey builds the terrain map on the ground display.

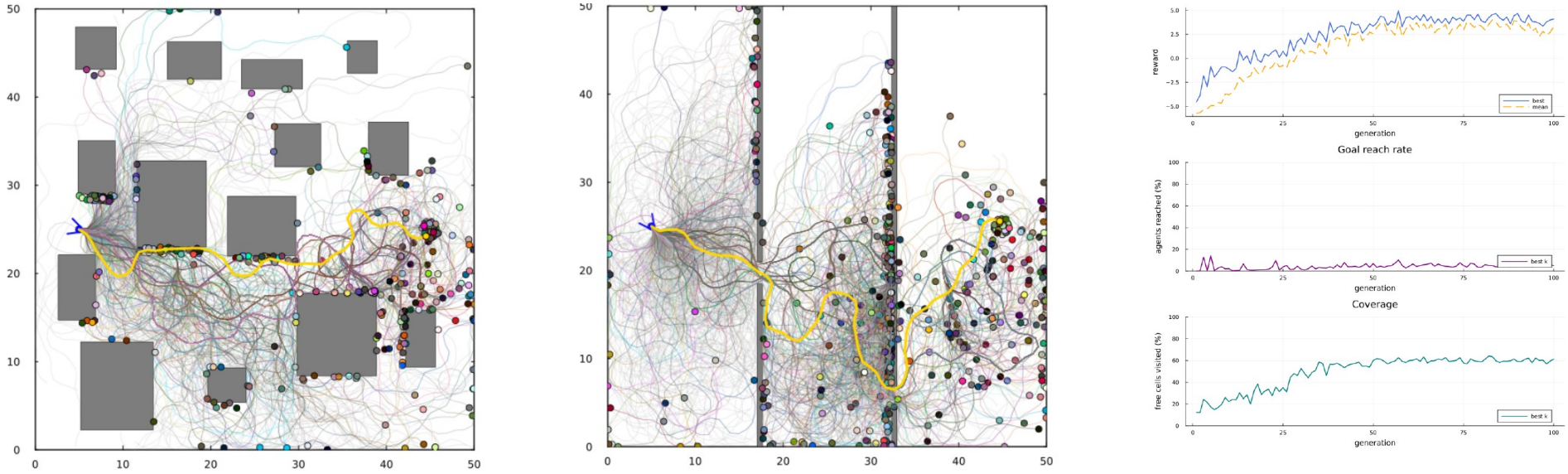
Control

- Cascaded PID loops: pitch angle to elevator, bank angle to ailerons, sideslip to differential tail deflection, and airspeed to throttle.
- Surface commands are divided by V^2 so the gains follow dynamic pressure, and a $1/\cos\phi$ term adds pitch in banked turns.
- Integrators and outputs are clamped to prevent windup, and a manual override mode resets the integrators.

Kino-PASTA

Multi-agent kinodynamic motion planning with a learned policy

Spring 2026 | ASEN 5264, Decision Making under Uncertainty (graduate)



Left: 500 agents running the trained policy through scattered obstacles and two narrow passages, best path in yellow. Right: training reward, goal-reach rate, and coverage over 100 generations.

Sampling-based planners grow a tree, and on a GPU that tree grows fast enough to run out of memory. Kino-PASTA replaces the tree with hundreds of independent agent paths that never branch, so memory grows linearly and the structure maps onto parallel hardware.

Each agent sees three ray casts, its own progress, and the direction to the goal. A shared neural network policy picks continue, backtrack, or teleport to a promising agent. We trained the policy in Julia with the Cross-Entropy Method. Agents that got through a narrow gap learned to flag themselves as teleport targets, which pulled other agents through behind them.

With Owen Kranz and Joe Krauskopf. I did much of the initial code development and tuning.

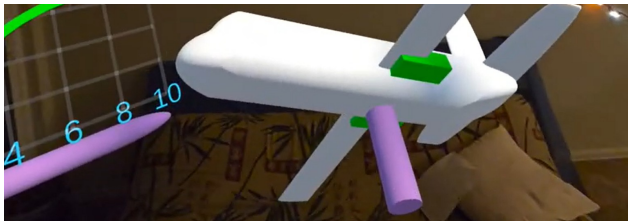
Planner	Env.	Success	Time (s)
Kino-PASTA	Scattered	100/100	0.028
	Narrow	97/100	0.081
Kino. RRT	Scattered	100/100	0.008
	Narrow	95/100	0.111

Mean time to first solution over 100 random environments per type, CPU only. RRT is faster in scattered fields; Kino-PASTA is about 30% faster through narrow passages.

XR Design & Fly

Mixed reality sandbox: reshape an airplane in your hands, then fly it

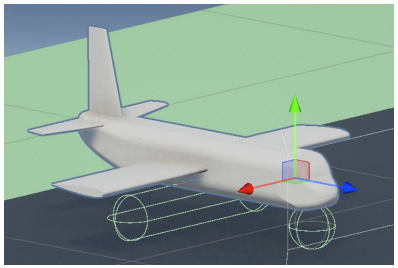
Aug – Dec 2025 | CSCI 4830, Intro to Mixed Reality



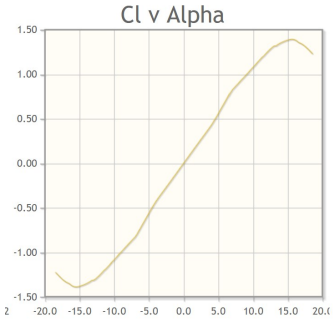
The editable design model in mixed reality.



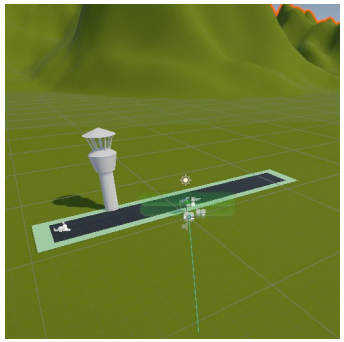
Mirrored plane and basic environment in the headset.



Aircraft model in Unity.



NACA 0012 lift data.



Terrain and ATC tower for visual scale.

Code: GitHub repository

The idea

Games like Kerbal Space Program got me into engineering. I wanted a design tool where changing a wing and flying it are one continuous motion, with physics honest enough to build real intuition.

Architecture

A design model the user grabs and resizes, mirrored by a physics-driven flight model. A camera renders to a virtual monitor for a cockpit view.

Flight model

- Left wing, right wing, horizontal tail, and vertical tail each apply their own forces to a rigid body, so pitch and yaw behavior come from the geometry. Roll is a commanded moment.
- Lift and drag come from analytic fits I made to NACA 0012 data, with the lift slope corrected for the aspect ratio the user edits.
- Mass and CG are computed from the size and position of each component, and tail incidence is set from the geometry.
- No rudder, adverse yaw, or sideslip effects, to keep it approachable for beginners.

Scaling the physics

Large airplanes appear to fly slower because gravity stays fixed while mass and aerodynamic forces scale differently with size. I added scale as its own dimension in the physics model: mass grows with the cube of scale and aerodynamic forces with the square, so a hand-sized model flies like a much larger aircraft. Scale is a control on an in-headset panel.

RC Aircraft

2022 | 150+ hours

High school project: the first airplane I designed, twin-engine balsa and birch

Design

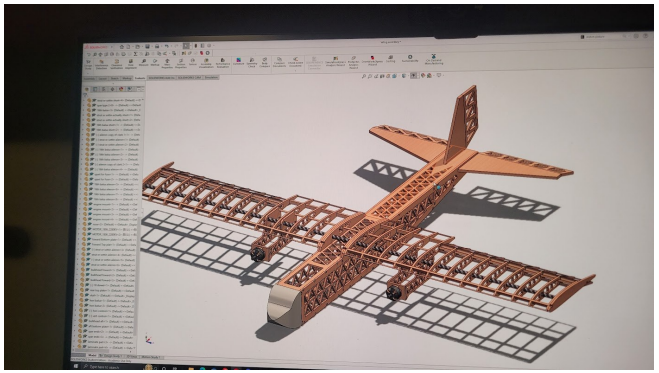
The goal was a model airplane built with computer-integrated design that could take off and land under its own power, with its flight characteristics predicted beforehand. I traded 3D printing, balsa, and foam and chose laser-cut 1/8" balsa with birch where the structure needed strength. I moved to a high wing for roll stability instead of dihedral, used a Clark Y airfoil, and estimated takeoff at about 6.2 m/s (13 mph) from the wing profile and measured weight.

Build

Cutting and assembly exposed problems I hadn't caught in CAD. I sheeted the trailing edge to stop the wing twisting, rescaled a vertical tail that was far too small, and added a laminated brace to carry wing loads into the fuselage. My dad helped me think through several of these.

First flight

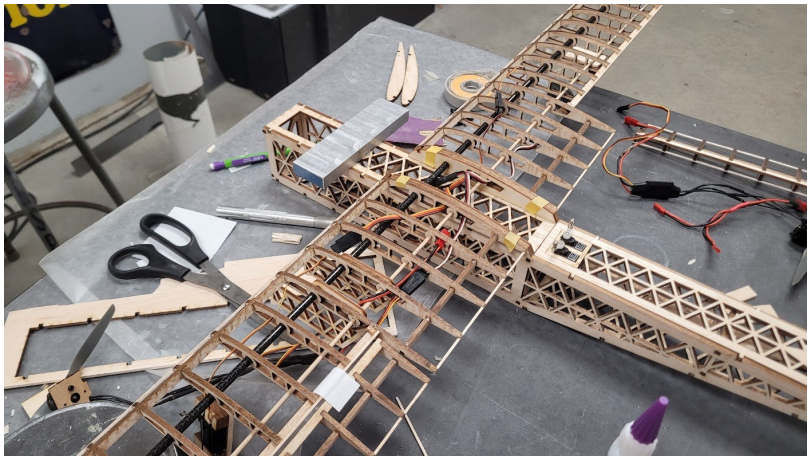
It flew well and was stable except in pitch. The CG felt aft in the air but balanced forward on the ground. My read at the time was that the boxy fuselage was acting as a lifting body and moving the center of pressure forward, so I added nose weight and cut openings in the fuselage covering. On the next flight it flew very well.



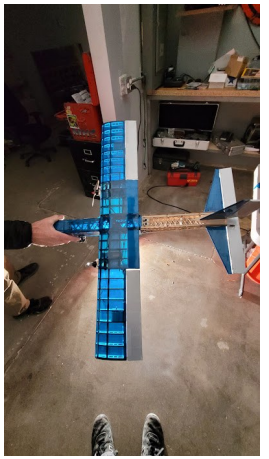
Full assembly in SolidWorks.



Finished frame before covering.



Wing and fuselage assembly.



Completed aircraft.